Hardware Implementation and PPA Evaluation of CXL-Inspired Switch Schedulers

Jay Chiang Yun-Chi Chou

ECE 509 Second Progress Report

Abstract

When several hosts share pooled CXL memory, their requests meet at shared switch ports, and the arbiter at each port decides whose memory access waits. We built a single-output, CXL-inspired switch in RTL with per-source FIFOs and a fixed output contract, and used it to compare four schedulers: first-come-first-served (FCFS), round-robin (RR), a scheduler inspired by FAI-CXL that classifies sources by the service they received over a long window, and a scheduler inspired by BLISS that blacklists a source after a short run of back-to-back services. This report describes the framework, the implementations, the benchmark we used to compare them, and our first measurements. With one always-busy source and three intermittent sources, the FAI-inspired scheduler lowers the 99th-percentile (P99) waiting time of the intermittent requests from 18.10 to 10.00 cycles for independent arrivals and from 33.45 to 21.05 cycles for bursts, compared with FCFS. Round-robin with a quantum of ten reaches 11.00 and 17.70 cycles, so it is better than FAI-inspired on bursts. The BLISS-inspired scheduler has the lowest tail for independent arrivals (1.15 cycles) but is almost as bad as FCFS for bursts (33.00 cycles). At four sources, the complete FAI-inspired design is 5.9% larger than FCFS and 39.3% larger than RR in mapped cell area. Neither history-based scheduler beats a simple round-robin on both arrival patterns, and round-robin is also the cheapest design.

1 Introduction

Both of us are interested in computer architecture, and especially in what happens between the chips rather than inside them. In a modern datacenter, the speed of a server depends more and more on its interconnects: how processors, accelerators, and memory are linked and how they share those links. Compute Express Link (CXL) caught our attention because it brings memory onto this kind of shared fabric. A processor can use memory that sits behind a CXL switch with ordinary loads and stores, and several servers can share the same pool of memory. That is a big change in how a server is put together, and it raised a question we wanted to answer for ourselves: once many hosts send memory requests through the same switch, who decides whose request goes first, and how much does that decision cost in hardware?

The FAI-CXL paper [6] gave us a concrete starting point. It proposes a scheduler that watches how much service each host has recently received and gives lightly served hosts a higher priority. The idea is appealing, but the paper evaluates it at the system level, and it leaves open how the scheduler behaves cycle by cycle and how much logic it adds compared with a simple arbiter. We decided to find out by building it. Our project implements a simplified CXL switch output in RTL, places several schedulers in exactly the same switch, and measures both the waiting time they produce and their area and power after synthesis.

In our first report we finished the FCFS and round-robin (RR) schedulers and described the plan. Since then we have built the common switch framework, integrated FCFS and RR into it, implemented the FAI-inspired scheduler, and run all of them on the same benchmark and synthesis flow. We also added a scheduler inspired by BLISS [7]. It uses service history like FAI, but over a much shorter time and with very little hardware, which helps us see whether FAI's behavior comes from using history at all or from the specific way FAI uses it.

Looking back at the first report, several parts were too brief for a reader who does not already know our design. We now explain why a CXL switch is worth studying in the first place, describe how the FAI scheduler and its Buffer class work before going into our implementation, keep every FAI-specific rule (including the service order and window updates) in one section, and define what we mean by PPA before showing any hardware numbers.

Throughout the report we focus on three questions. How does each policy choose among the queues that compete for one output? How does that choice affect a lightly loaded host that shares the output with a heavily loaded one? And how much complete hardware does each policy need?

2 Background and Motivation

2.1 CXL and Memory Pooling

CXL runs on the same physical layer as PCIe but adds protocols for coherent devices and for memory. Its memory protocol, CXL.mem, lets a processor read and write device-attached memory with normal cacheable loads and stores, so software sees it simply as more memory [1]. CXL 2.0 added switches, so several hosts can now reach a common set of memory devices. This is what makes memory pooling possible: instead of giving every server enough DRAM for its worst case, a provider can put part of the memory in a

shared pool and lend it to whichever server needs it. Pond found that a large share of server memory in today's clusters is never used and showed that a pool can recover much of it, as long as the extra latency stays small [2]. Not everyone agrees that pooling pays off [3], but the debate itself shows how much the latency of the shared path matters.

What interested us is a side effect of pooling. When the memory becomes shared, the path to it becomes shared too. Requests from different hosts meet at a switch port, on a link, and at a memory device, as in Figure 1(a). Our project zooms in on one of those meeting points: a single switch output with one queue per host.

2.2 Why a CXL Switch and Not an Ethernet Switch?

At first glance a CXL switch output is not very different from an Ethernet switch output. Both have queues, a limited output rate, an arbiter, and backpressure from downstream, and many of the scheduling ideas we use, such as round-robin and fair queueing, come from networking [4, 5]. We do not claim that switch scheduling is a new problem. What made the CXL case interesting to us is what a delayed request costs and how congestion is handled.

A request in a CXL memory switch is usually a cache miss. The core that issued it, or the instructions that depend on it, must wait for the data. CXL memory is already slower than local DRAM [2], so even a few extra cycles of queueing at a shared port turn directly into stalled processor time. An Ethernet packet, on the other hand, usually belongs to a flow that the end host can buffer and pace, and the transport layer can hide a few microseconds of queueing. This is why we look at the tail of each request's waiting time (P99) and not only at throughput.

Congestion also behaves differently. CXL links use credit-based flow control and do not drop requests [1], so when an output is busy, requests stay queued and the stall spreads back toward the hosts. An Ethernet switch can drop or mark packets and let the senders slow down. On a lossless memory path, the arbiter is one of the few places where a heavy host can be kept from crowding out a light one. Our framework models this with a ready/valid handshake and FIFOs that refuse input when full.

Two more properties make the problem a good fit for a hardware study. Memory requests move about one cache line each, so counting requests is a fair measure of service, while network fair queueing has to track variable packet sizes [5]. And because the arbiter sits on the memory path and must decide every cycle, every bit of extra state, such as arrival timestamps or per-host counters, costs area and power at every port. For these reasons we always report a scheduler's waiting time together with its complete hardware cost.

2.3 The FAI-CXL Scheduler

FAI-CXL (Fairness-Aware Interleaved CXL) is a pooled-memory design with three parts: a pool manager that allocates memory to hosts, weighted interleaving that spreads each host's cache lines over several memory devices, and a scheduler that arbitrates between hosts [6]. We only reconstruct the scheduler.

The scheduler works in fixed windows. During a window it counts how many requests each host had served, $B[h]$, and the total $S = \sum_h B[h]$. At the end of the window it compares each host with the average S/N. A host that received less than 30% of the average gets evidence to move up (UP). A host that received more than the average gets evidence to move down (DOWN). Everything else is KEEP. For example, with $N = 4$ and $S = 1000$, the average is 250. A host with 700 services is DOWN, a host with 10 services is UP, and a host with 150 is KEEP. Note that the classifier sees service that was actually delivered, not demand. A host with a long queue that was served little looks the same as a host that sent little.

Hosts are placed in a High class or a Low class, and the arbiter alternates between the two classes. Without any smoothing, a host could change class after every window. Suppose a heavy host is moved to Low. In the next window it gets less service, so its count drops and it may look like an UP candidate again. The classes would then oscillate, and so would the latency seen by everyone else. FAI-CXL adds a Buffer class to prevent this. A host that has new evidence first enters Buffer, and only a second window with the same evidence completes the move. A single unusual window therefore does not change the class. The price is that a real change in behavior takes two windows to show up.

The FAI-CXL paper describes Buffer in two ways that lead to different hardware. The text (Section IV-C) says a Buffer host keeps its previous priority for one window. Algorithm 2 instead assigns the new target priority as soon as the host enters Buffer [6]. We implement the first reading as our main design and keep the second as an alternative (Section 5.5). Because of these choices, and because our arbiter is single-output, we call our design FAI-inspired.

2.4 The BLISS Scheduler

The Blacklisting Memory Scheduler (BLISS) comes from DRAM memory controllers, where several applications on a multicore chip share one controller [7]. Earlier fair schedulers such as ATLAS and TCM rank every application by its memory behavior and order requests by that rank [8, 9]. They work well, but the ranking logic is large and slow, because ranks have to be recomputed and every request has to be compared by rank. BLISS asks whether a much simpler rule is enough.

The rule comes from one observation: an application that hurts others usually gets many requests served back to back, because it always has requests ready (often row-buffer hits). BLISS counts how many requests in a row come from the same application. When the count passes a threshold of four, that application goes on a blacklist. The controller then picks requests in this order: requests from applications that are not blacklisted, then row-buffer hits, then older requests. The whole blacklist is cleared every 10,000 cycles, so no application is held back for long. The hardware is one blacklist bit per application, one counter, and the ID of the last application served. The paper reports performance and fairness

Field	Width	Purpose
Tag	32 bits	Matches requests in the checker
Address	32 bits	Request address
Opcode	1 bit	Read or write
Length	8 bits	Carried along; does not change service cost
Payload	32 bits	Request data
Total	105 bits	One output slot per descriptor

Table 1: Descriptor stored in every FIFO entry.

	FCFS	RR10 / RR1
Selection	Oldest head overall	Next nonempty source after pointer
Policy state	Timestamp per entry	Pointer and quota
Ties	Source ID	Cyclic order
On fire	Pop	Pop and decrement quota
Main cost	Timestamp storage and comparators	Very little; waiting depends on Q

Table 2: FCFS and RR at a glance.

similar to or better than the ranking schedulers at a much lower hardware cost [7].

We include BLISS for three reasons. First, it protects light requesters from a heavy one by looking at recent service, which is also FAI's goal. Second, its history is very short (a streak of a few services, cleared every 10,000 cycles), while FAI's is very long (a 5M-cycle window plus the Buffer step). Comparing the two shows how much the length of the history matters. Third, BLISS was designed to be cheap in hardware, and our first report already borrowed its practice of measuring scheduler cost in synthesized logic.

2.5 Fairness Reference

Stall-Time Fair Memory scheduling made the point that fairness needs an explicit reference, such as performance when running alone [10]. In our setting, equal service per class and equal service per source are different goals, and we report which one each rule gives.

3 Common CXL-Inspired Switch Framework

3.1 Datapath

Figure 1(b) shows the slice we implemented. It has N request sources, a depth-D FIFO for each source, a policy module, and a common output controller. We use $N = 4$, 8, and 16 with $D = 16$. Each cycle the policy picks one nonempty FIFO head, and at most one request leaves the switch. The memory side is a testbench sink that can assert or drop `ready`. The framework abstracts a CXL egress port into request descriptors. It does not include CXL flit formatting, link credits, coherence, multiple memory outputs, or DRAM timing.

3.2 Handshake and Backpressure

All interfaces use ready/valid. A transfer happens only when $fire = out_valid \wedge out_ready$. Only a fire pops a FIFO, uses up scheduler quota, or increments a service counter. If the sink is not ready, the output descriptor, the chosen source, and the grant information (for example, which FAI class it was charged to) stay fixed until the sink accepts it. New arrivals and class changes cannot replace an offer that is already on the output. Other FIFOs can still accept input during the stall.

Admission and arbitration use the FIFO state at the start of the cycle. A full FIFO refuses input even if its head leaves on the same edge, and a newly accepted request can be selected one cycle later. We fixed these rules so that no policy gets a different answer for the same arrival pattern.

3.3 What Is Shared and What Each Policy Pays For

All designs use the same descriptor, FIFO depth, admission rules, and output controller. Each design also contains the extra state its policy needs and nothing else. FCFS and FAI-inspired store a 32-bit arrival timestamp per entry. RR stores no timestamps. Adding unused timestamps to RR would make it look more expensive, and leaving them out of FCFS would hide the cost of ordering by age, so we compare complete designs.

4 FCFS and Round-Robin

4.1 FCFS

FCFS writes a 32-bit timestamp into each FIFO entry when the request is accepted. Every cycle it compares the timestamps of the nonempty heads and picks the oldest, breaking same-cycle ties by source ID. Since each FIFO is already in arrival order, comparing heads is enough. The comparison is modular and assumes that no request stays in the switch for 2^{31} cycles or more, which makes wraparound safe. The tag and the timestamp have different jobs: the tag identifies a request for checking, and the timestamp sets its order.

4.2 Round-Robin

RR keeps a pointer and a remaining quota. From the pointer it finds the next nonempty source and serves it for up to Q transfers, moving on early if that source empties. Empty sources are skipped, and if only one source has work, RR keeps serving it with no idle cycle. A downstream stall does not use up quota because nothing was transferred. Our main setting is $Q = 10$ (RR10). We also run $Q = 1$ (RR1) to show how much the quantum matters. A larger quantum gives longer bursts from one source, which could help memory locality in a real system, but a request that arrives during another source's visit waits longer. Our model measures the waiting and does not model locality.

The standalone FCFS and RR modules were verified for the first report. The new work is running them inside the shared slice under the same traffic and sink behavior as the FAI-inspired design.

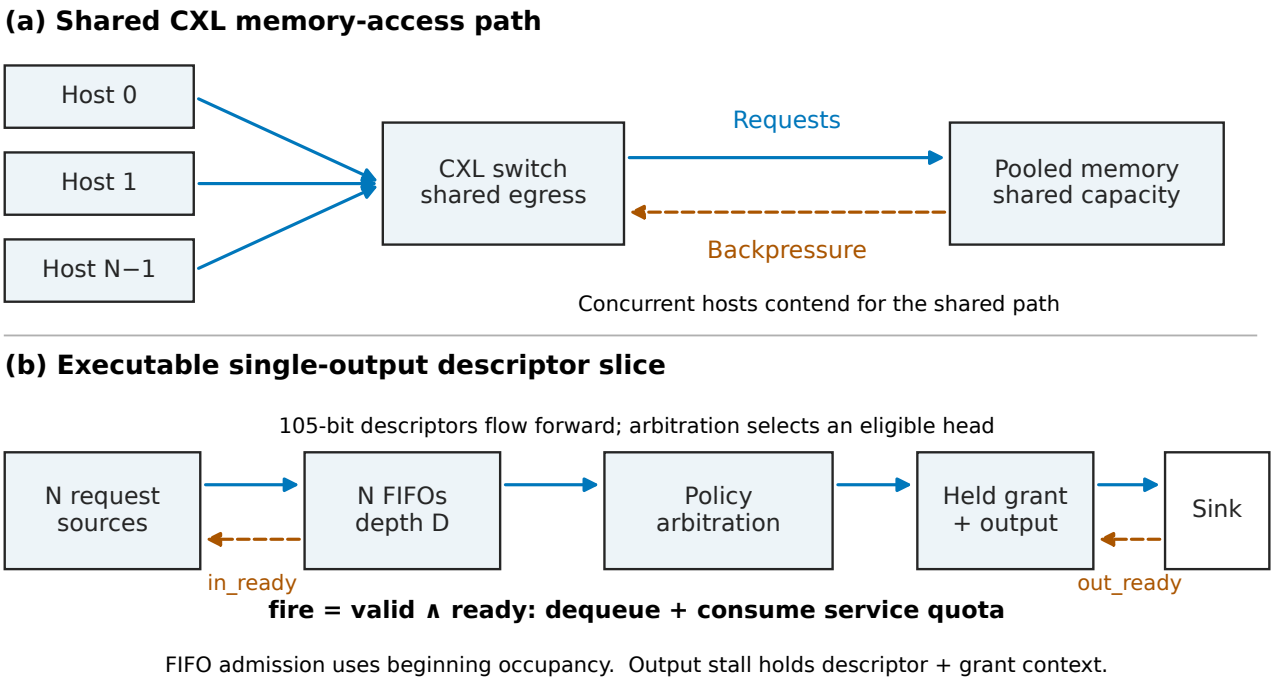


Figure 1: (a) Motivating system: several hosts reach pooled memory through a shared CXL switch egress. We do not implement the pool or the CXL protocol. (b) What we built: N sources, one FIFO per source, a pluggable arbiter, and a held output. Every 105-bit descriptor costs one output slot.

5 FAI-Inspired Scheduler Implementation

This section covers everything that is specific to FAI, including the service order and window boundaries that were in Section 4.5 of our first report.

5.1 Counters and Thresholds

For each source h, $B[h]$ counts fires in the current window, and S counts all fires. The average uses all N configured sources, including idle ones. To avoid division, we compare

$$\begin{aligned} \text{UP}: &\quad 10NB[h] < 3S, \\ \text{DOWN}: &\quad NB[h] > S, \\ \text{KEEP}: &\quad \text{otherwise (including equality).} \end{aligned} \tag{1}$$

The window W is 5,000,000 cycles and keeps counting during stalls. The counters are 23 bits wide, and the products are computed in 32 bits so that they do not overflow at $N = 16$. A window with $S = 0$ cancels any pending change and keeps the current class.

5.2 Buffer States

Each source has a 2-bit state: H, B_H, L, or B_L (Table 3, Figure 2). H and B_H are served as High, and L and B_L as Low. All sources start in H. One DOWN window moves H to B_H, and the source is still served as High. A second DOWN moves it to L. If KEEP or UP comes instead, it goes back to H and its service never changed. The Low side works the same way with UP. This is the smoothing described in Section 2.3. In our benchmark the Heavy source is DOWN in every window, so it becomes B_H after the first window and L after the second.

State	Served as	UP	KEEP	DOWN
H	High	H	H	B_H
B_H	High	H	H	L
L	Low	B_L	L	L
B_L	Low	H	L	L

Table 3: Next state at the end of a window (main design).

5.3 How the Classes Are Served

The arbiter alternates between a High visit and a Low visit, and each visit allows up to ten fires. Within a class it picks the oldest head using the FCFS timestamp rule. If the current class has no work, it gives up the rest of its quota, and if only one class has work, that class keeps the output without an idle cycle. This rule gives two busy classes the same number of fires. It does not give each source the same number. With one Low source and three High sources all backlogged, the Low source gets about half the output and each High source about one sixth (Section 8.4).

5.4 Window Boundaries and Stalls

At the last edge of a window, a fire on that edge is counted in $B[h]$ and S first. Then all sources are classified, the counters are cleared, and the new classes take effect in the next cycle. If the output is stalled at that moment, the offer keeps its

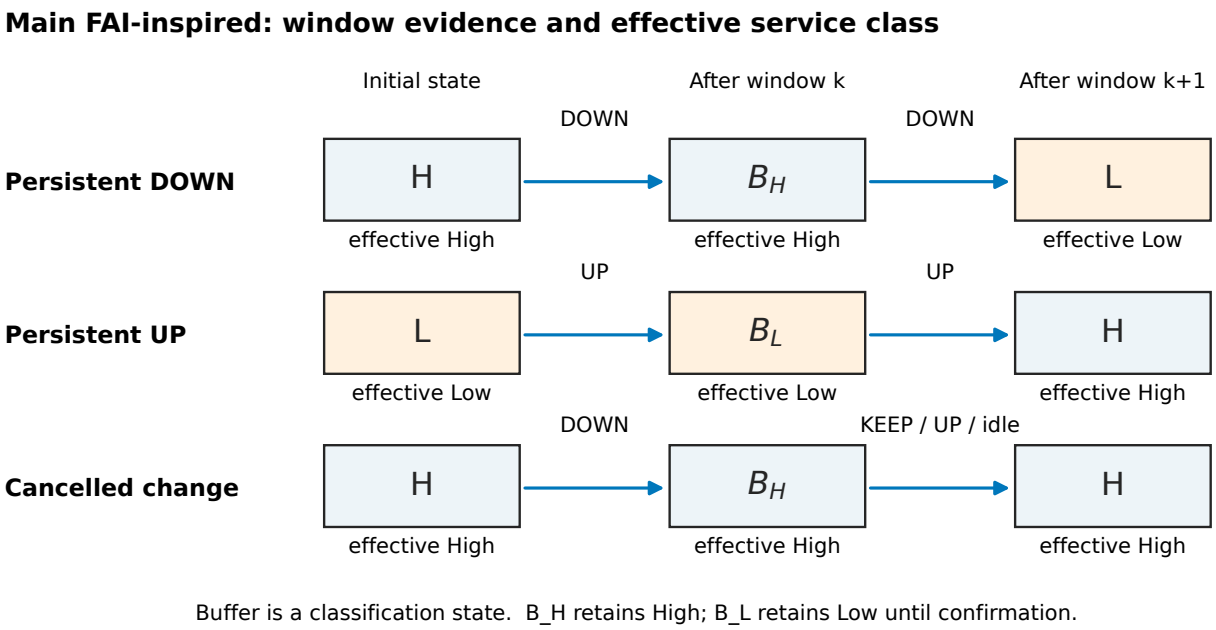


Figure 2: Buffer states in our main design. A move between High and Low needs two windows in a row with the same evidence. B_H is still served as High and B_L as Low. KEEP, opposite evidence, or an idle window cancels the pending move. Buffer is a classifier state and holds no requests.

original source and class. Its service is counted in the window in which it actually fires, and the quota is charged to the class that was captured when it was selected. The class turn and remaining quota carry over into the new window.

5.5 Alternative Buffer Interpretation

For the Algorithm 2 reading (FAI-ImmBuf), a source entering Buffer is served in its new target class right away, and stays in Buffer on KEEP or an idle window. It uses the same thresholds and the same 10/10 alternation. We ran it through the same benchmark to see whether the choice matters (Sections 8.1 and 8.2).

6 BLISS-Inspired Scheduler Implementation

Our switch has no DRAM, so there are no row-buffer hits. We keep the blacklist and replace the row-hit rule with the oldest-first rule from FCFS. Each cycle the arbiter picks the oldest head among nonempty sources that are not blacklisted. If every nonempty source is blacklisted, it picks the oldest head among all of them.

The scheduler stores the ID of the last source that fired and a streak counter. When a fire comes from a different source, the counter is reset to zero. When it comes from the same source, the counter goes up by one, and if the new value is above four, that source is blacklisted and the counter is cleared. Counting from a source change, a source that keeps being served is blacklisted on its sixth service in a row. A timer clears all blacklist bits every 10,000 cycles, and if a clear and a new blacklist happen in the same cycle, the clear wins. As in the other designs, a stalled offer is held, and the counter and blacklist only change on a real fire.

Compared with FCFS, the extra state is one blacklist bit per source, the streak counter, the last source ID, and the clear timer. It still needs per-entry timestamps for the oldest-first step. We call it BLISS-inspired because the row-hit rule is removed and the policy works on whole descriptors instead of DRAM commands.

7 Common Platform and Measurement Method

Every policy runs in the same switch slice with the same descriptor, FIFO depth, admission rules, output capacity, and hold behavior (Section 3). For a given seed, every policy also receives exactly the same generated requests and the same sink-ready sequence. Once the queues start to fill, the policies can accept and serve requests at different times, but any difference in the results comes from the scheduling rule and not from the inputs. This section explains what traffic we use, which groups of runs we made, how we measure waiting, service, and fairness, and how we check the hardware.

7.1 Traffic, Policies, and Test Cohorts

We do not have CXL application traces, so we built a synthetic workload around the case that motivated the project: one host that always has work and several hosts that send requests only now and then. The main runs use $N = 4$ and $D = 16$. Source 0 is the *Heavy* source and generates one request every cycle, so its FIFO is always full. Sources 1–3 are *Sparse*. Each run has three phases.

The run starts with 10 million cycles of *training*, which covers two FAI windows. This lets the FAI classifier settle (Heavy ends in L, Sparse in H), so we measure trained be-

Cohort	What is run
RTL confirmation (20 seeds, 529–548)	All six policies, Arms A and B, always-ready sink
RTL exploration (5 seeds, 509–513)	One factor at a time: RR quantum 1–16; FAI High quota 5/10/20; FIFO depth 8/16/32; 32-request bursts; sink ready 75% (random or periodic 48/64); $N = 8$ and 16. FCFS, RR, and FAI-inspired only
Fixed-class model (20 seeds, 6201–6220)	One Low and three High sources, all backlogged, 20,000 cycles; measures per-source share
Buffer transition model ($W = 256$)	Model with and without the Buffer step; 78 transition-only replays per design (noise pulses and real role changes)
Hardware	FCFS, RR10, FAI-inspired at $N = 4$, 8, 16 for area; three trained seeds per policy for $N = 4$ power

Table 4: Test cohorts. Confirmation results come with paired 95% confidence intervals; exploration and model results are descriptive.

havior instead of the start-up period in which every source is High. Next comes the *measurement* phase, which uses one of two Sparse arrival patterns. In *Arm A (independent arrivals)*, each Sparse source generates a request with probability 1/128 in every cycle for 100,000 cycles, so requests usually arrive alone. In *Arm B (bursts)*, each Sparse source sends eight requests at once, at a random offset, in each of 100 blocks of 1,024 cycles, so several requests compete at the same moment. Heavy keeps generating in both arms. Arm A asks whether a policy protects isolated requests, and Arm B asks whether it still does so when the light sources become busy for a short time. Finally, a 1,024-cycle *guard* keeps Heavy running, and a *drain* completes all outstanding requests so that every measured Sparse request is counted.

The six RTL policies are FCFS, RR1, RR10, FAI-inspired (our main FAI design), FAI-ImmBuf (the alternative Buffer reading), and BLISS-inspired. Table 4 lists the groups of runs. The *confirmation* cohort uses 20 seeds (529–548), the sink is always ready, and it is the only cohort for which we report confidence intervals. The *exploration* cohort uses a separate set of five seeds (509–513) and changes one factor at a time. We keep the two seed sets apart so that we do not tune parameters on the same runs we use to draw conclusions. Two small model studies explain mechanisms. One fixes the classes and measures each source's share, and the other replays short class transitions to see how the Buffer reacts. The hardware cohort covers synthesis and power.

7.2 Waiting, Service, and Fairness

For each request i we record the cycle in which it was generated (g_i), accepted into its FIFO (a_i), and sent out (d_i):

$$L_{GF,i} = d_i - g_i = (a_i - g_i) + (d_i - a_i). \tag{2}$$

The first part, generation to acceptance, is time spent waiting outside a full FIFO. The second part, acceptance to fire, is time spent in the switch queue. The identity holds for each request, but the P99 of the two parts does not have to add up to the P99 of the total.

Our main metric is *Sparse P99*. For each run we take the nearest-rank P99 of $d - g$ over all Sparse requests, then average these per-run values across seeds. This is not the same as one P99 over all requests from all seeds pooled together. To compare two policies, we pair their runs by seed, take the difference, and use 10,000 bootstrap resamples for a 95% confidence interval. The seed is the independent unit, not the individual request, since requests in one run are not independent of each other.

Waiting time alone does not show how the output is divided. We also record the number of fires per measured cycle (throughput) and the number of fires per source (service share). Fairness only makes sense against a reference [10], and different rules aim at different references. RR aims at equal service per active source, while FAI's 10/10 rule aims at equal service per class. We therefore report the actual per-source shares instead of a single fairness index. A raw index such as Jain's can be low just because one source generates most of the work, which in our Heavy/Sparse workload is true by design.

7.3 Hardware and Implementation Checks

We check every design against an independent Python reference model with a cocotb scoreboard. The models use plain software queues and sorting, not a copy of the RTL selector. They compare every transfer cycle by cycle and check that only a nonempty source is served, that at most one request leaves per cycle, that no request is lost or duplicated, that each source's order is kept, and that a stalled offer does not change. For FAI we also test threshold equality, S accounting, reset, and fires and stalls on the window edge. At the end of each run we confirm that the numbers of generated, accepted, and fired requests match.

For hardware, all designs go through the same synthesis and timing flow with the same settings (Section 9.2). The area numbers include the complete design (FIFOs, timestamps, policy logic, and shared control), not only the policy block. We also run RTL-to-netlist checks to confirm that each synthesized netlist behaves like its RTL, so that the area and power we report belong to the design we simulated.

8 Results

8.1 Main Comparison

Table 5 and Figure 3 show the 20-seed results. Compared with FCFS, the FAI-inspired scheduler lowers Sparse P99 by 8.10 cycles in Arm A and by 12.40 cycles in Arm B. FCFS does badly because a Sparse request that arrives behind up to 16 queued Heavy requests has to wait for all of the older ones. The FAI-inspired scheduler puts Heavy in Low after training, so Sparse requests get their own High visits.

Round-robin is the more interesting comparison. With independent arrivals, FAI-inspired and RR10 are close (10.00 vs. 11.00 cycles). With bursts, FAI-inspired is 3.35 cycles worse than RR10, with a paired 95% interval of [2.25, 4.45], so the difference is not noise. Between the two round-robin settings, RR1 has lower tails in both arms because it never lets one source keep the output for more than one transfer;

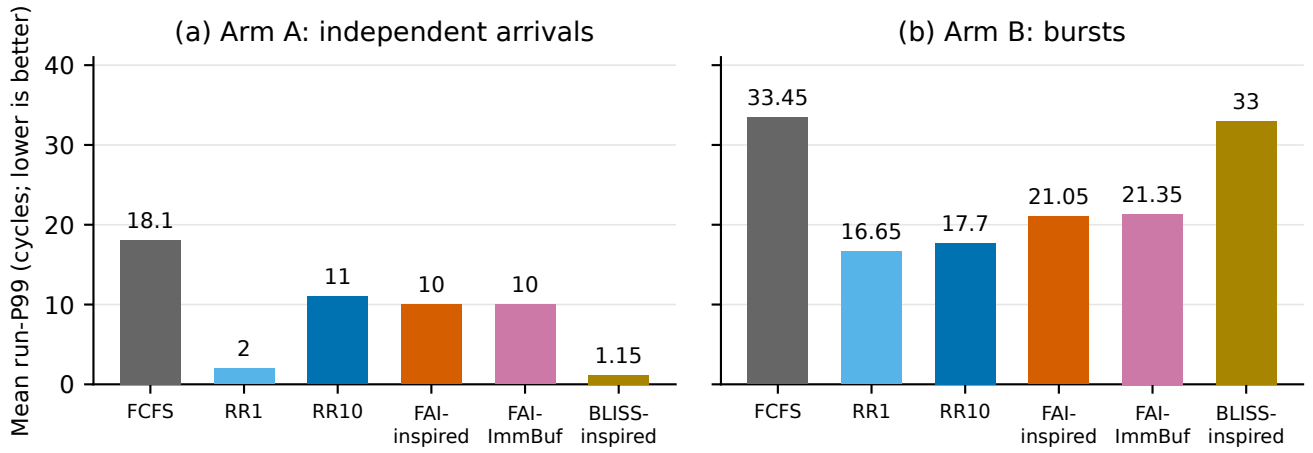


Figure 3: RTL confirmation: mean per-run Sparse P99 (generation to fire) in cycles, 20 seeds (529–548), $N = 4$, $D = 16$, W = 5M cycles, after two training windows. (a) Arm A: each Sparse source sends single requests at random (probability 1/128 per cycle). (b) Arm B: each Sparse source sends an 8-request burst in every 1,024-cycle block. Lower is better.

Policy	Arm A (cycles)	Arm B (cycles)
FCFS	18.10	33.45
RR1	2.00	16.65
RR10	11.00	17.70
FAI-inspired (main)	10.00	21.05
FAI-ImmBuf (alternative)	10.00	21.35
BLISS-inspired	1.15	33.00

Table 5: Sparse P99, mean over 20 seeds.

across all policies, it has the lowest tail in Arm B (16.65 cycles), while BLISS-inspired is lowest in Arm A (1.15 cycles, Section 8.3). The two Buffer interpretations give almost the same result (21.05 vs. 21.35 in Arm B). In a separate check where the Heavy and Sparse roles swap during a run, the alternative interpretation changed classes after one window (5M cycles) instead of two (10M), but that faster reaction did not lower Sparse waiting.

8.2 Does the Buffer Smooth Class Changes?

The benchmark above runs long enough for the classes to settle, so it does not show the Buffer at work. We checked it with the transition model, which is a software model rather than RTL and uses a short window ($W = 256$) so that many window boundaries can be replayed. We compared our main design with a version without the Buffer step (Main-1W, which moves class after one window) and with FAI-ImmBuf. When a source shows one window of unusual service (a noise pulse), our main design makes no effective class change, while Main-1W and FAI-ImmBuf each flip the class twice, once away and once back. When a source really changes behavior, our main design settles in 448–704 cycles, compared with 192–448 cycles for Main-1W. This is the trade-off described in Section 2.3: the Buffer ignores one-window noise, but a real change takes about one extra window to take effect. As noted in Section 8.1, faster reaction did not lower Sparse waiting in our role-switch runs.

8.3 BLISS-Inspired: Isolated Arrivals vs. Bursts

BLISS-inspired has the best Arm A tail of all the policies (1.15 cycles), lower even than RR1. Its Arm B tail (33.00 cycles) is close to FCFS (33.45). The two arms exercise different parts of the rule.

In Arm A, Heavy is served back to back most of the time, so it is blacklisted after six fires and again shortly after every clear. A Sparse request that arrives alone comes from a source that is not blacklisted, so it is chosen at the next decision. Requests only wait longer right after a clear, when Heavy is briefly off the blacklist and its older requests win on age. This is the situation BLISS was designed for.

In Arm B, each Sparse source receives eight requests at once. Our current explanation, which we still need to confirm with waveforms, is that a burst makes a Sparse source look like a heavy one. If a bursting source gets six services in a row, it is blacklisted too. Once Heavy and the bursting sources are all blacklisted, the arbiter falls back to oldest-first among them, which is plain FCFS, and the Sparse requests wait behind older Heavy requests again. The 10,000-cycle clear period is also about ten burst blocks long, so a Sparse source can stay blacklisted through several later bursts. An Arm B tail almost equal to FCFS fits this explanation.

So BLISS-inspired reacts much faster than FAI-inspired, finding Heavy within a few cycles instead of two 5M-cycle windows. But a short streak cannot tell a heavy source from a light source that happens to burst, and FAI's long window can. In our tests neither history length handled both arrival patterns well.

8.4 Why FAI-Inspired Loses on Bursts

The answer is in how the classes are served. After training, Heavy is the only Low source and the three Sparse sources share High. The 10/10 rule gives each class half of the output whenever both have work, so Heavy alone gets half while the three Sparse sources share the other half. We checked this with fixed classes and all four sources backlogged for 20,000

cycles: the Low source received 49.9975% of the service and each High source about 16.67%. When all three Sparse sources burst at the same time, they compete for ten slots per High visit, and then must wait for a full Low visit by Heavy. In a replay of seed 509 we saw no Sparse FIFO overflow, but seven High visits ran out of quota while High requests were still queued. RR10 has no such grouping: each Sparse source gets its own visit.

So the classifier does its job. It finds the heavy source and moves it to Low. The problem is that equal quotas per class are not equal service per source when the classes have different sizes. This points to the allocation rule as the part to change next, not the classifier.

8.5 Sensitivity and Exploration

Figure 4 changes one parameter at a time using the five exploration seeds. These numbers are descriptive and are not directly comparable with the 20-seed means.

RR quantum. RR's Arm A tail grows roughly with Q, from about 2 cycles at $Q = 1$ to about 17 at $Q = 16$, because an arriving request may wait for a whole visit by Heavy. The burst tail stays near 16–17 cycles up to $Q = 10$ and then rises.

FAI class quota. Raising the High quota from 5 to 10 to 20 (Low fixed at 10) lowers the Arm B tail from 32.2 to 20.8 to 17.0 cycles, while Arm A stays at 10. This supports the explanation above: giving the larger class more slots fixes most of the burst problem.

FIFO depth. With $D = 8$, 16, and 32, FCFS's Arm A tail goes from 10.0 to 18.2 to 35.2 cycles, because a deeper FIFO holds more old Heavy requests ahead of a new Sparse one. FAI-inspired stays at 10 and RR10 at 11, since neither serves strictly by age across sources.

Harder conditions. With 32-request bursts, the Arm B tails are 88.8 cycles for FAI-inspired, 74.4 for RR10, and 87.0 for FCFS. With $N = 8$ and $N = 16$, FAI-inspired is still above RR10 in Arm B (27.60 and 34.80 vs. 21.00 and 24.40). When the sink is ready only 75% of the time, FAI-inspired's A/B tails rise to 17.2/32.6 cycles with random readiness and 26.0/34.0 with periodic readiness (48 of every 64 cycles). The average readiness is the same, but the pattern changes the tail, so we will report the pattern along with the average.

9 Hardware Cost (PPA)

9.1 What PPA Means Here

PPA stands for power, performance, and area, the three numbers usually used to compare digital hardware designs. Here they mean the following. **Area** is the total area of the standard cells that synthesis maps the design to; a larger area means more silicon per switch port. **Performance** is timing: whether the design still works at a given clock period, which static timing analysis (STA) checks by finding the slowest path between registers. **Power** is the average power the gate-level netlist draws while it runs our benchmark traffic, estimated from simulated switching activity. These are hardware measurements. Request waiting time (Section 8) is a separate behavioral metric.

All three numbers are *pre-layout*: they come from synthesis before placement and routing, so wire and clock-tree effects are only estimated. Absolute values may change after layout. The comparison is still meaningful because every design goes through the same flow with the same settings.

9.2 Setup

We use Synopsys Design Compiler and PrimeTime W-2024.09 with the SAED32 32 nm regular-threshold library at the typical corner (0.78 V, 25 °C) and a 25 ns clock. Wire-load model, clock uncertainty, I/O delays, and output loads are the same for all designs. The area always covers the complete design: descriptor FIFOs, any timestamps, the policy logic, and the shared output control. We synthesized FCFS, RR10, and FAI-inspired at $N = 4$, 8, and 16. The BLISS-inspired design runs in the RTL benchmark, but we have not synthesized it yet, so it is missing from the tables below.

9.3 Area

Table 6 breaks down the $N = 4$ designs. The descriptor FIFOs are the same in all three. FCFS and FAI-inspired both pay about 13,500 units for timestamps. The FAI-specific part (counters, threshold comparators, class states, and class arbitration) adds about 6,000 units of policy logic over FCFS. Overall, FAI-inspired is 5.9% larger than FCFS and 39.3% larger than RR10. Across $N = 4$, 8, and 16, it stays 38.2–39.3% larger than RR10. The storage grows with N: at $N = 16$, the 256 FIFO entries hold 26,880 descriptor bits, and FCFS and FAI-inspired need another 8,192 timestamp bits. RR is cheaper both because its arbiter is simple and because it stores no timestamps.

9.4 Power

Table 7 gives power estimates from the trained benchmark runs, averaged over three seeds. RR10 uses about 2.56 mW. FCFS uses 3.36 mW, about 31% more, and FAI-inspired 3.47 mW, about 35% more than RR10. The ranking follows area. These estimates use zero-delay gate activity and ideal clocks with no back-annotated delays, and the FCFS and RR runs reported some library table-range warnings. We use them to rank designs, not as absolute chip power.

10 Summary and Next Steps

The framework, the schedulers, the benchmark, and the area and power flow are now in place. Our main finding so far is that the FAI-inspired scheduler fixes FCFS's problem with a heavy source, but in our workloads it does not beat RR10, and it costs about 39% more area. The cause is the class-level allocation, not the classifier. The BLISS-inspired scheduler has the opposite trade-off: its short history protects isolated Sparse requests very well, but bursts make it behave like FCFS.

Next, we plan to (1) confirm the results at $N = 8$ and $N = 16$ with 20 seeds, (2) study how the window length affects how quickly FAI reacts when traffic changes, (3) try allocation rules that divide service per source rather than per class, (4) synthesize the BLISS-inspired design and check

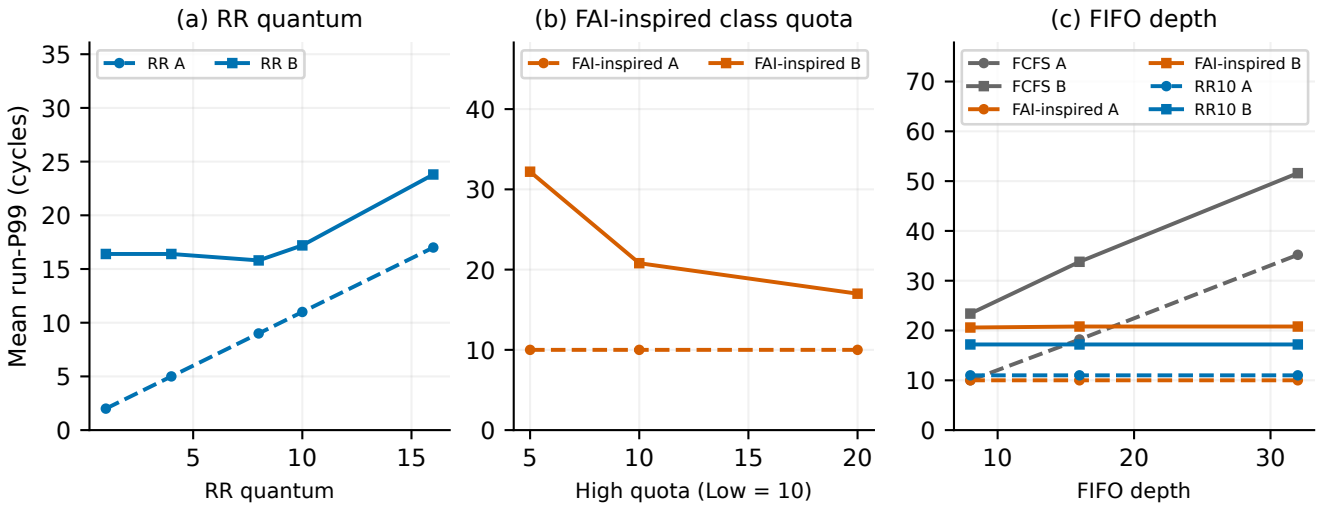


Figure 4: One-factor sensitivity runs (five seeds, 509–513, no confidence intervals). Mean per-run Sparse P99 in cycles; solid lines are Arm B (bursts) and dashed lines are Arm A. (a) RR quantum. (b) FAI High-class quota with Low fixed at 10. (c) FIFO depth.

$N = 4$ design	Descriptor FIFOs	Timestamps	Policy logic	Common logic	Total area
FCFS	44,404.041	13,532.660	1,131.957	44,901.400	103,970.058
RR10	44,404.041	0	263.039	34,380.092	79,047.172
FAI-inspired	44,404.041	13,532.660	7,143.988	45,009.157	110,089.846

Table 6: Complete mapped cell area at $N = 4$ (SAED32 library area units), split into non-overlapping parts. Common logic is the same RTL in all designs but is optimized differently by synthesis.

$N = 4$ design	Arm A (mW)	Arm B (mW)
FCFS	3.356	3.356
RR10	2.560	2.560
FAI-inspired	3.467	3.466

Table 7: Pre-layout power estimates, mean of three trained seeds.

our explanation of its burst behavior with waveforms, and (5) add timing slack and post-layout numbers where possible. Linking the descriptor model to a full CXL memory model with real response latency is outside what we can do in this project.

11 Changes from the First Report

This report builds on our first progress report. Its sections were *1 Motivation*, *2 Related Work and Project Difference*, *3 Key Challenges*, *4 Solution Sketch* (4.1–4.5), *5 Implementation and Evaluation Plan* (5.1–5.4), and *6 Progress Made So Far*. Table 8 lists every section of this report and how it relates to the first one. "New" means the material did not exist before. "Heavily edited" means the topic was in the first report but the text was largely rewritten or extended. The first report's Key Challenges section was removed, because those challenges (equal-cost service, stall handling, window-edge timing, and fair hardware comparison) are now answered by the concrete rules in Sections 3–5 and 7.

AI use acknowledgment. We used AI tools to help write RTL verification scripts, to organize our experiments, and to edit this report. All numbers in this report come from our own RTL simulations and synthesis runs, and we checked them against the raw results.

References

[1] D. Das Sharma, R. Blankenship, and D. Berger, "An Introduction to the Compute Express Link (CXL) Interconnect," *ACM Computing Surveys*, vol. 56, no. 11, 2024. doi: 10.1145/3669900.

[2] H. Li et al., "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms," in *Proc. ASPLOS*, 2023. arXiv:2203.00241.

[3] P. Levis, K. Lin, and A. Tai, "A Case Against CXL Memory Pooling," in *Proc. ACM HotNets*, 2023, pp. 18–24. doi: 10.1145/3626111.3628195.

[4] P. Gupta and N. McKeown, "Designing and Implementing a Fast Crossbar Scheduler," *IEEE Micro*, vol. 19, no. 1, pp. 20–28, 1999. doi: 10.1109/40.748793.

[5] M. Shreedhar and G. Varghese, "Efficient Fair Queuing Using Deficit Round-Robin," *IEEE/ACM Trans. Networking*, vol. 4, no. 3, pp. 375–385, 1996.

[6] J. Li, Y. Liu, Y. Gu, and C. Wu, "FAI-CXL: An Efficient Hardware-Accelerated Fairness-Aware CXL Memory Pool Management with Fine-Grained Cacheline-Level Interleaving," in *Proc. IEEE ICPADS*, 2025. doi: 10.1109/ICPADS67057.2025.11322896.

[7] L. Subramanian, D. Lee, V. Seshadri, H. Rastogi, and O. Mutlu, "The Blacklisting Memory Scheduler: Achieving High

This report	Status	Notes
1 Introduction	Heavily edited	From old §1; adds our motivation, the FAI-CXL starting point, and progress since the first report
2.1 CXL and Memory Pooling	New	CXL background
2.2 Why a CXL Switch and Not an Ethernet Switch?	New	Why CXL switch scheduling is worth studying
2.3 The FAI-CXL Scheduler	New	Expands the short FAI paragraph in old §2; Buffer smoothing and a worked example
2.4 The BLISS Scheduler	New	New related work and reason for including BLISS
2.5 Fairness Reference	Heavily edited	From old §2
3 Common Switch Framework (3.1–3.3)	Heavily edited	From old §4.1–4.2; adds Figure 1, Table 1, and 3.3
4 FCFS and Round-Robin	Heavily edited	From old §4.3; describes the implemented RTL
5.1–5.2 FAI counters and Buffer states	Heavily edited	From old §4.4; adds Figure 2
5.3–5.5 FAI service order, window edges, alternative Buffer	Heavily edited	Moved from old §4.5 into the FAI section
6 BLISS-Inspired Implementation	New	
7 Common Platform and Measurement Method	Heavily edited	From old §5.1–5.2; plan replaced by the cohorts actually run (Table 4)
8 Results (8.1–8.5)	New	First latency results, Buffer test, BLISS and sensitivity analysis
9.1 What PPA Means Here	New	Defines power, performance, and area
9.2 Setup	Heavily edited	From old §5.3–5.4
9.3–9.4 Area and Power	New	First hardware results
10 Summary and Next Steps	Heavily edited	Replaces old §6
11 Changes from the First Report	New	This section

Table 8: Delta over the first progress report.

Performance and Fairness at Low Cost," in *Proc. IEEE ICCD*, 2014, pp. 8–15. doi: 10.1109/ICCD.2014.6974655.

[8] Y. Kim, D. Han, O. Mutlu, and M. Harchol-Balter, "ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers," in *Proc. IEEE HPCA*, 2010, pp. 1–12. doi: 10.1109/HPCA.2010.5416658.

[9] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter, "Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior," in *Proc. IEEE/ACM MICRO*, 2010, pp. 65–76. doi: 10.1109/MICRO.2010.51.

[10] O. Mutlu and T. Moscibroda, "Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors," in *Proc. IEEE/ACM MICRO*, 2007, pp. 146–160. doi: 10.1109/MICRO.2007.21.